

Лабораторная работа № 14

Отчет

Гайдук Софья Сергеевна

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Контрольные вопросы	10
4	Выводы	12
	Список литературы	13

Список иллюстраций

2.1	image1	6
2.2	image2	7
2.3	image3	7
2.4	image4	8
2.5	image5	8
2.6	image6	9
2.7	image7	9

Список таблиц

1 Цель работы

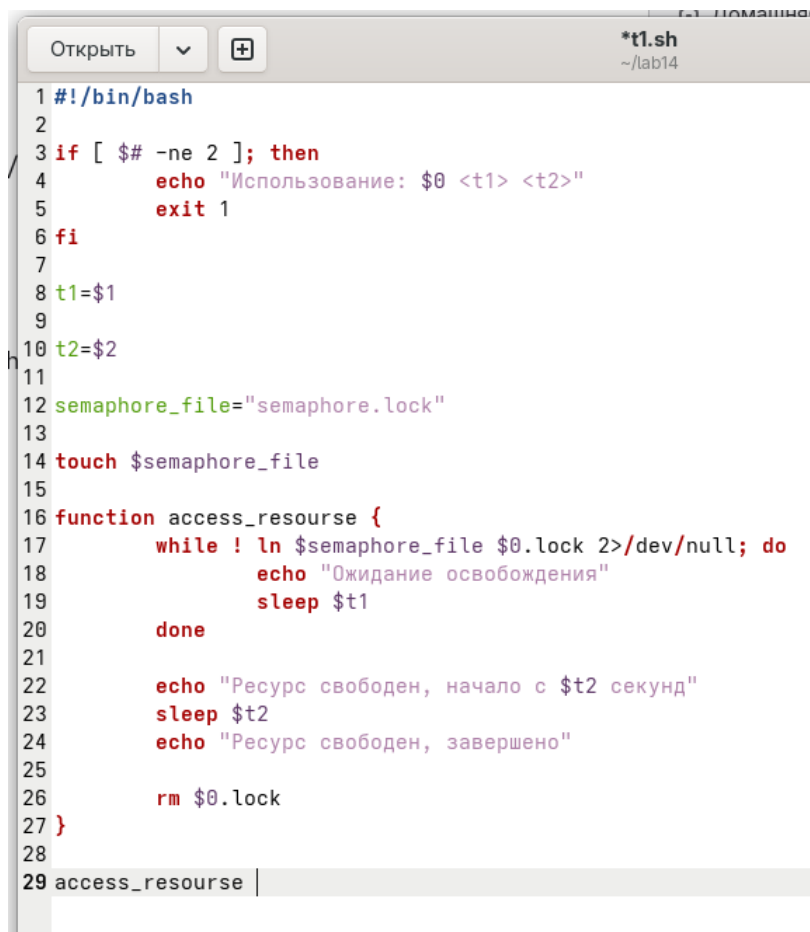
Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов

2 Выполнение лабораторной работы

Написать командный файл, реализующий упрощённый механизм семафоров. Командный файл должен в течение некоторого времени t_1 дожидаться освобождения ресурса, выдавая об этом сообщение, а дождавшись его освобождения, использовать его в течение некоторого времени $t_2 < t_1$, также выдавая информацию о том, что ресурс используется соответствующим командным файлом (процессом) (рис. 2.1, рис. 2.2, рис. 2.3).

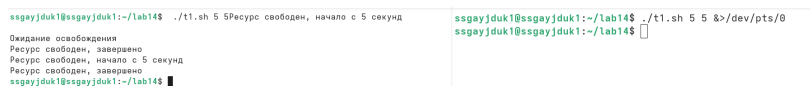
```
ssgayjduk1@ssgayjduk1:~$ touch t1.sh
ssgayjduk1@ssgayjduk1:~$ mkdir lab14
ssgayjduk1@ssgayjduk1:~$ █
```

Рисунок 2.1: image1



```
1 #!/bin/bash
2
3 if [ $# -ne 2 ]; then
4     echo "Использование: $0 <t1> <t2>"
5     exit 1
6 fi
7
8 t1=$1
9
10 t2=$2
11
12 semaphore_file="semaphore.lock"
13
14 touch $semaphore_file
15
16 function access_resource {
17     while ! ln $semaphore_file $0.lock 2>/dev/null; do
18         echo "Ожидание освобождения"
19         sleep $t1
20     done
21
22     echo "Ресурс свободен, начало с $t2 секунд"
23     sleep $t2
24     echo "Ресурс свободен, завершено"
25
26     rm $0.lock
27 }
28
29 access_resource
```

Рисунок 2.2: image2



```
sagayjdk1@sagayjdk1:~/lab14$ ./t1.sh 5 5 Ресурс свободен, начало с 5 секунд
Ожидание освобождения
Ресурс свободен, завершено
Ресурс свободен, начало с 5 секунд
Ресурс свободен, завершено
sagayjdk1@sagayjdk1:~/lab14$

sagayjdk1@sagayjdk1:~/lab14$ ./t1.sh 5 5 &>/dev/pts/0
sagayjdk1@sagayjdk1:~/lab14$
```

Рисунок 2.3: image3

Реализовать команду `map` с помощью командного файла (рис. 2.4, рис. 2.5).

```
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Использование: $0 <название_команды>"
    exit 1
fi

command_name=$1

man_directory="/usr/share/man/man1"

if [ -f "$man_directory/$command_name.1.gz" ]; then
    zcat "$man_directory/$command_name.1.gz" | less
else
    echo "Справка для команды '$command_name' не найдена"
fi
```

Рисунок 2.4: image4

```
ssgayjduk1@ssgayjduk1:~/lab14$ ./t2.sh mnbvc
Справка для команды 'mnbvc' не найдена
ssgayjduk1@ssgayjduk1:~/lab14$ ./t2.sh ls
ssdaviduk1@ssdaviduk1:~/lab14$
```

Рисунок 2.5: image5

Используя встроенную переменную \$RANDOM, напишите командный файл, генерирующий случайную последовательность букв латинского алфавита (рис. 2.6, рис. 2.7).

Открыть ▾  t3.sh
~/lab14

```
#!/bin/bash

generate_random_letter() {
    random_nuber=$((RANDOM % 26))

    letter=$(printf \\$(printf '%03o' $((65 + $random_nuber))))

    echo -n "$letter"
}

random_sequence=""

for ((i=0; i<10; i++)); do
    random_sequence="$random_sequence$(generate_random_letter)"
done

echo "Случайная последовательность $random_sequence"
```

Рисунок 2.6: image6

```
ssgayjduk1@ssgayjduk1:~/lab14$ ./t3.sh
Случайная последовательность CXNRXODWWK
ssgayjduk1@ssgayjduk1:~/lab14$ ./t3.sh
Случайная последовательность XPSWTKHZSD
ssgayjduk1@ssgayjduk1:~/lab14$ ./t3.sh
Случайная последовательность VCVWLAQSLR
ssgayjduk1@ssgayjduk1:~/lab14$
```

Рисунок 2.7: image7

3 Контрольные вопросы

1. Найдите синтаксическую ошибку в следующей строке: `while [$1 != «exit»]`
 - Синтаксическая ошибка — отсутствуют пробелы после `[` и перед `]`. Правильно:
`while [ «$1» != «exit» ]`
2. Как объединить (конкатенация) несколько строк в одну?
 - Объединение (конкатенация) строк в `bash` — простой записью переменных подряд: `result=«str1str2»` или с разделителем: `result=«$str1 $str2»`. Также можно использовать `+=`
3. Найдите информацию об утилите `seq`. Какими иными способами можно реализовать её функционал при программировании на `bash`?
 - Утилита `seq` и её альтернативы — `seq` генерирует последовательности чисел. Альтернативы в `bash`:
 - `{1..10}` — для фиксированных пределов
 - `for ((i=1; i<=N; i++))` — C-подобный цикл для переменных пределов
4. Какой результат даст вычисление выражения `$((10/3))`?
 - Результат `$((10/3))` — будет 3, так как в `bash` арифметика целочисленная
5. Укажите кратко основные отличия командной оболочки `zsh` от `bash`.
 - Основные отличия `zsh` от `bash`:

- zsh: лучшая автодополнение, подсветка синтаксиса, проверка орфографии
- bash: простота, POSIX-совместимость, максимальная распространённость

6. Проверьте, верен ли синтаксис данной конструкции `for ((a=1; a <= LIMIT; a++))`

- Синтаксис `for ((a=1; a <= LIMIT; a++))` — верен, это стандартный C-подобный цикл в `bash`. Пробелы вокруг `<=` допустимы

7. Сравните язык `bash` с какими-либо языками программирования. Какие преимущества у `bash` по сравнению с ними? Какие недостатки?

- Преимущества `bash`: вездесущность (есть в любом Linux), простое объединение команд, не требует установки
- Недостатки `bash`: множество сложных мест (`IFS`, `CDPATH` и др.), сложность написания надёжных скриптов более 100 строк, неудобная обработка путей

4 Выводы

Мы изучили основы программирования в оболочке ОС UNIX. Научились писать более сложные командные файлы с использованием логических управляющих конструкций и циклов

Список литературы

1.Kulyabov. Лабораторная работа № 14. Программирование в командном процессе ОС UNIX. Расширенное программирование. RUDN